

# CHEEM: Continual Learning by Reuse, New, Adapt and Skip – A Hierarchical Exploration-Exploitation Approach

Chinmay Savadikar  
 North Carolina State University  
 csavadi@ncsu.edu

Michelle Dai  
 Johns Hopkins University  
 mdai12@jh.edu

Tianfu Wu  
 North Carolina State University  
 tianfu\_wu@ncsu.edu

## Abstract

*To effectively manage the complexities of real-world dynamic environments, continual learning must incrementally acquire, update, and accumulate knowledge from a stream of tasks of different nature—without suffering from catastrophic forgetting of prior knowledge. While this capability is innate to human cognition, it remains a significant challenge for modern deep learning systems. At the heart of this challenge lies the stability-plasticity dilemma: the need to balance leveraging prior knowledge, integrating novel information, and allocating model capacity adaptively based on task complexity and synergy. In this paper, we propose a novel exemplar-free class-incremental continual learning (ExfCCL) framework that addresses these issues through a **Hierarchical Exploration-Exploitation (HEE)** approach. The core of our method is a HEE-guided efficient neural architecture search (HEE-NAS) that enables a learning-to-adapt backbone via four primitive operations—reuse, new, adapt, and skip—thereby serving as an internal memory that dynamically updates selected components across streaming tasks. To address the task ID inference problem in ExfCCL, we exploit an external memory of task centroids proposed in the prior art. We term our method **CHEEM** (Continual Hierarchical-Exploration-Exploitation Memory). CHEEM is evaluated on the challenging MTIL and VDD benchmarks using both Tiny and Base Vision Transformers and a proposed holistic **Figure-of-Merit (FoM)** metric. It significantly outperforms state-of-the-art prompting-based continual learning methods, closely approaching full fine-tuning upper bounds. Furthermore, it learns adaptive model structures tailored to individual tasks in a semantically meaningful way.<sup>1</sup>*

## 1. Introduction

Developing continual learning machines is a key objective in Artificial Intelligence (AI), aiming to replicate human-

like adaptability and the ability to learn-to-learn, enabling proficiency in streaming tasks. Despite their advances, state-of-the-art Deep Neural Networks (DNNs) still lack true biological intelligence in the realm of continual learning from streaming tasks in dynamic environments, which requires the continual acquisition, update, and accumulation of knowledge while mitigating catastrophic forgetting of previous tasks [26, 38], referring to the stability-plasticity trade-off.

Recently, continual learning using Vision Transformers (ViTs) [5] has witnessed promising progress - particularly in the Exemplar-free Class Incremental Continual Learning (ExfCCL) setting, where the raw data (or latent features of samples) of old tasks are not available in learning a new task, and task IDs of testing samples are unknown at inference. Fig. 1 shows an overview of the prior art in ExfCCL. There are four key aspects of interest in this paper:

**i) What benchmarks to use?** Many existing works focus on unrealistic balanced benchmarks such as Split ImageNet-R benchmark [44] with equal number of classes per streaming tasks, as well as equal number of training images. In the literature, challenging benchmarks such as VDD [33] that has significantly varying number of classes and training images per task has been used in early works like [20] in the task-incremental setting. More recently, benchmarks of similar nature to VDD such as MTIL [52] have been proposed to test ExfCCL in more practical real-world scenarios (see examples in Fig. 3a). We adopt these benchmarks and focus on both VDD and MTIL in this paper.

**ii) How to adapt ViT backbone to address the stability and plasticity challenge?** Task 1 (e.g., ImageNet-1k) is typically assumed to train a ViT sufficiently well [36, 40, 43, 44]. There are two main backbone adaptation strategies: *prompting-based methods* that retain strong stability but lack plasticity by keeping the ViT backbone frozen throughout all streaming tasks, and resorting to learning task-specific prompts or prefixes [36, 40, 43, 44], and *parameter-tuning methods* that maintain plasticity but lack

<sup>1</sup>Code: <https://github.com/savadikarc/cheem>.

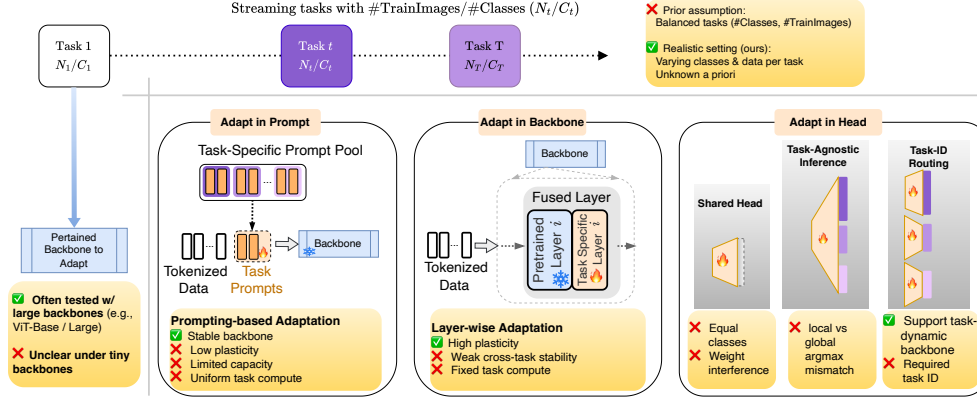


Figure 1. **Taxonomy of ExfCCL design choices.** Existing methods typically adapt either the *prompt* [36, 40, 43, 44] or the *backbone* (often layer-wise [8, 12, 13, 25, 28, 45, 47]), together with different strategies in the *head* adaptation, each with trade-offs in stability, plasticity, inference assumptions, and compute adaptivity. See text for detail.

task-synergy stability by adapting all layers in a ViT using either parameter-masking [25, 45, 47], output addition [8, 12, 13, 28], or adapter tuning [22, 46]. More importantly, both strategies often overlook and fail to address a critical aspect: **how to adapt computation to be task-difficulty/synergy-aware?** Prompting-based methods emphasize *Reuse* of every layer in a ViT, while parameter-tuning methods utilize *Adapt* of every layer. Both keep ViT backbone architecture frozen, and thus they either retain the same computation across tasks or increase it when new task-specific prompts appended in the input, regardless of task difficulty. There are two critical components underexplored in the prior art: One is a *Skip* operation that bypasses certain layers in ViT when a task is a relatively easy one, enabling smaller backbones for easy tasks. The other is a *New* operation that introduces a totally new layer to substitute an existing one when a task is significantly different from previous ones, enabling novel information integration quicker and better than *Adapt* (that is constrained by existing pretrained weights). In this paper, we propose a method of learning the four operations (*Reuse*, *Adapt*, *New* & *Skip*) to best balance the stability and plasticity in addressing challenges of ExfCCL in the VDD and MTIL benchmarks, which leads to task-difficulty/synergy-aware dynamic backbones.

**iii) How to learn task head classifiers?** There are three different designs with corresponding underlying assumptions. A *shared task head* is used when all tasks are assumed to have the exact same number of classes in those unreal balanced benchmarks, which also entails strong weight consolidation in training to overcome catastrophic forgetting in ExfCCL. A *task-agnostic head* can eliminate the need of unreal assumption of equal class numbers across tasks and is designed to handle no explicit task-ID inference. However, it suffers from the discrepancy between training and inference. During training, task ID is known, only local-argmax is used in computing the loss, while in inference, without task ID of a testing sample, the global-argmax is used to predict the class. The consensus between

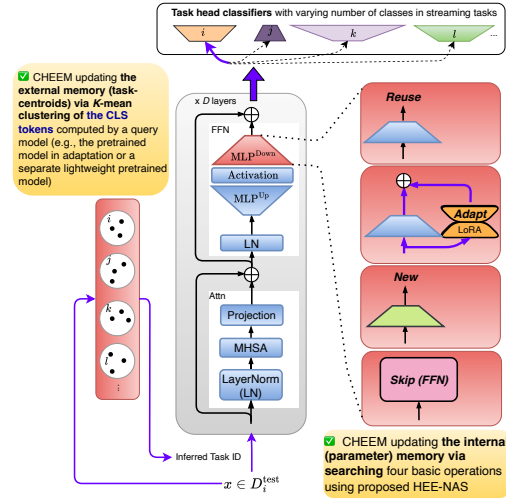


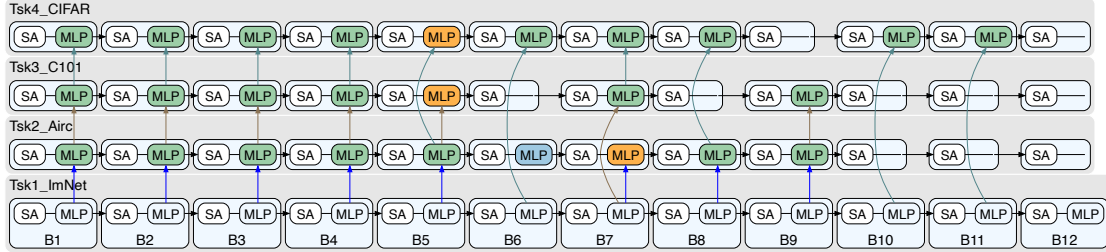
Figure 2. **Illustration of the proposed CHEEM.** A pretrained and frozen ViT model such as ViT-Base [5] or DEiT-Tiny [39] is structurally and dynamically updated to learn internal (parameter) memory for streaming tasks in continual learning, and is also used in maintaining the external task-centroid memory of CHEEM. CHEEM learns the internal parameter memory for a selected component such as the  $MLP^{Down}$  layer. We also test placing CHEEM at the projection layer in the ‘Attn’ block.

local-argmax and global-argmax is very difficult to preserve in ExfCCL, for which we show a theoretical justification in the supplementary (Sec. I). Our experimental results also empirically reflect the difficulty of this task-agnostic head design. *Task-specific heads* do not suffer from those issues, but entail explicit task ID inference of testing samples [40]. In this paper, we adopt task-specific head design, which is consistent with our proposed dynamic backbones, both entailing task ID inference.

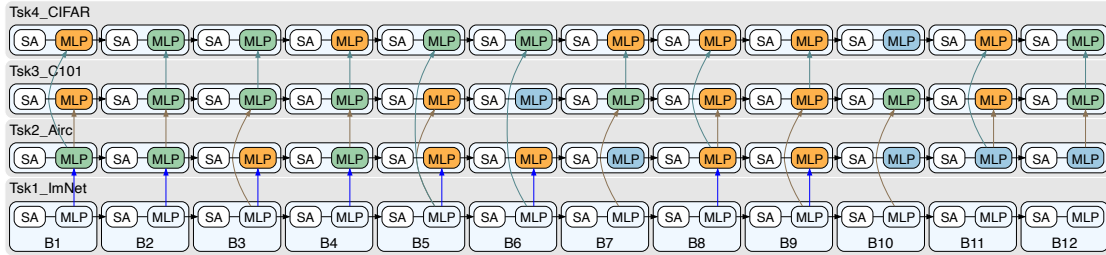
**iv) What capacity of ViT backbone to start with?** Most of existing work start with high-capacity ViT backbones such as ViT-Base/Large, which leaves ExfCCL with tiny backbones underexplored. ExfCCL with tiny backbones is useful in practice for the deployment of continual learning on edge devices. ExfCCL with tiny backbones also challenges the prompting-based methods since they are



(a) The MTIL benchmark [52] consisting of tasks of different nature with #training images/#classes significantly varying across different tasks.



(b) From ViT-Base trained on Tsk1\_ImNet (with blocks B1 to B12), our CHEEM learns sensible task-tailored models that reflect the task complexity. For example, when learning Caltech 101 (Tsk3\_C101), CHEEM learns to Skip 5 MLP blocks and Reuse most of the architecture. On the contrary, when learning FGVC Aircraft (Tsk1\_Airc), which is a more complex task with larger shift from ImageNet due to its fine-grained nature, CHEEM learns to Adapt the ImageNet parameters in Block 7, adds a New operation in Block 6, and Skips the last 3 MLP blocks. See text for details. Full structure reproduced in Fig. 7 in the supplementary.



(c) From DEiT-Tiny trained on Tsk1\_ImNet (with blocks B1 to B12), our CHEEM learns to use multiple Adapt and New operations, without Skip operations selected, sensibly different from those with more Skip and less New operations learned based on the stronger ViT-Base model. Full structure reproduced in Fig. 9 in the supplementary.

Figure 3. Examples of CHEEM learning task-tailored models.

upper-bounded by the capacity of the backbone, and advocates the need of introducing New operations. We test both base and tiny models with different yet meaningful task-tailored backbones learned (Fig. 3b and 3c).

Fig. 2 illustrates our proposed method. We view ExfCCL from continual memory perspective, consisting of our proposed novel internal parameter memory learning and external task centroid memory learning adopted from the prior art [40]. We term our proposed method **CHEEM** (*Continual Hierarchical-Exploration-Exploitation Memory*) in which a new task learns to automatically reuse/adapt modules from previous similar tasks, to introduce new modules when needed, or to skip some modules when it appears to be an easier task (see Figs. 3b and 3c). We propose a hierarchical exploration-exploitation (HEE) sampling based neural architecture search (NAS) method for learning the internal memory.

To ensure NAS is computationally efficient, and retain

the stability of the backbone to account for tasks in streams that have little training data, we select two components in a ViT block: the down projection layer ( $\text{MLP}^{\text{Down}}$ ) in the FFN and the projection layer after the MHSA, to be plastic to maintain the internal parameter memory in a compute-budget controllable way using:

- **Reuse**: Facilitates similar tasks sharing layers for knowledge transfer in continual learning.
- **New**: Explores new features for handling tasks that are dissimilar to previous tasks. It enables learning-to-grow the backbone to be skilled at streaming tasks.
- **Adapt**: Utilizes LoRA [15], inducing task synergies in ExfCCL in a parameter-efficient way.
- **Skip**: Skips the entire FFN block (when the  $\text{MLP}^{\text{Down}}$  is used) or the entire MHSA block accordingly. It can thus induce much simpler backbones for relatively easier tasks in a learning-to-prune manner, especially when a strong backbone such as ViT-Base is used (e.g., from ImageNet to MNIST).





the Supernet training (the parameter estimation of updating CHEEM), and iii) the target network selection and finetuning (the consolidation of the CHEEM for the task  $t$ ).

### 3.2. Supernet Construction

For clarity, we consider how the space of MoEs of the internal memory is constructed at a single layer  $l$  for a new task with CHEEM placed at the MLP<sup>Down</sup> (projection) layer, assuming the current memory consists of two experts,  $\{E_l^{(1)}, E_l^{(2)}\}$  (Fig. 4, left). The Supernet is constructed via:

- **Reuse**: Uses the MLP<sup>Down</sup> (projection) layer from an old task for the new task unchanged, exploiting task synergies during learning.
- **Adapt**: Introduces a new lightweight LoRA [15] component, e.g.,  $\theta_l^{(3,)} = \theta_l^{(2,)} + B_l \cdot A_l$ , where  $B_l$  and  $A_l$  are low-rank parameter matrices.
- **New**: Adds a new MLP<sup>Down</sup> (projection) layer, which enables the model to handle corner cases and novel situations.
- **Skip**: Skips the entire FFN (MHSA) block, which encourages dynamically adjusting the model complexity based on the task complexity.

The bottom of Fig. 4 shows the search space. The Supernet is constructed by reusing and adapting each existing expert at layer  $l$ , and adding a new and a skip expert. The newly added adapt ( $B_l, A_l$ ) by LoRA and projection layers will be trained from scratch using the data of a new task only. The right-top of Fig. 4 shows the Adapt operation on top of  $E_l^{(2,)}$  is learned and added,  $E_l^{(3,)} = (\theta_l^{(3,)}, \mu_l^3)$  where  $\mu_l^3$  is the mean CLS token pooled for the task 3.

### 3.3. Supernet Training via HEE-NAS

To train the Supernet constructed for a new task  $t$ , we build on the efficient SPOS method [14]. The vanilla SPOS trains a single-path sub-network from the Supernet by sampling an expert at every layer in each mini-batch. One key aspect is the sampling strategy. The vanilla SPOS method uses uniform sampling (i.e., the *pure exploration* (PE) strategy, Fig. 5 top). We propose an exploitation strategy (Fig. 5 bottom), which utilizes a hierarchical sampling method that forms the categorical distribution over the operations in the search space **explicitly based on task synergies computed based on the pooled task-specific CLS tokens**.

Consider a new task  $t$  with the training dataset  $D_t^{train}$ , with the current supernet consisting of  $t - 1$  task-specific target networks, we first run inference of the  $t - 1$  target networks on  $D_t^{train}$  to pool initial CLS tokens for each expert, e.g.,  $\mu_l^{1 \rightarrow 3}$  and  $\mu_l^{2 \rightarrow 3}$  in the bottom of Fig. 4. Consider one expert  $E_l^{(i,j)}$  at the  $l$ -th layer which is shared by two previous tasks  $i$  and  $j$  with their mean CLS tokens  $\mu_l^i$  and  $\mu_l^j$  respectively, we have the pooled CLS tokens for the current task  $t$ ,  $\mu_l^{i \rightarrow t}$  and  $\mu_l^{j \rightarrow t}$ , computed accordingly. The task

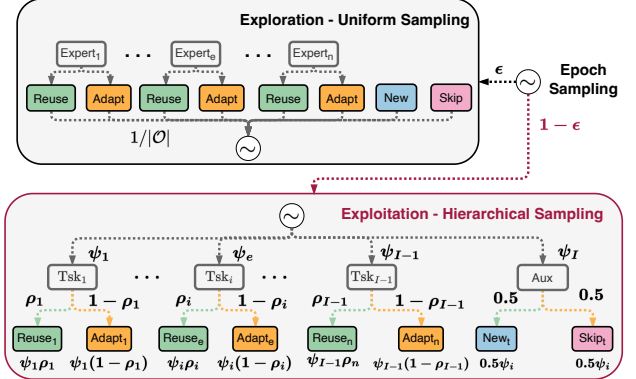


Figure 5. Illustration of the proposed HEE sampling based NAS. It integrates the vanilla exploration strategy (top) and the proposed exploitation strategy (bottom) with an epoch-wise scheduling.

similarity is computed by,

$$S_l^{i,t} = \text{NormCosine}(\mu_l^i, \mu_l^{i \rightarrow t}), \quad (2)$$

where  $\text{NormCosine}(\cdot, \cdot)$  is the Normalized Cosine Similarity, which is calculated by scaling the Cosine Similarity score between  $-1$  and  $1$  using the minimum and the maximum Cosine Similarity scores from all the experts in all the MHSA blocks of the ViT. This normalization is necessary to increase the difference in magnitudes of the similarities between tasks, which results in better Expert sampling distributions during the sampling process in our experiments. The task similarity score will be used in sampling the Reuse and Adapt operations.

For the new task  $t$ , we also have the New expert and the Skip expert at each layer  $l$ , for which we do not have similarity scores. Instead, we introduce an auxiliary expert, Aux (see the bottom of Fig. 5) which gives equally-likely chance to select the New expert or the Skip expert once sampled in NAS. For the Aux expert itself, the similarity score between it and the new task  $t$  is specified by,

$$S_l^{aux,t} = -\frac{t-1}{\max_{i=1}^{t-1} S_l^{i,t}}, \quad (3)$$

which intuitively means we probabilistically resort to the New operation or the Skip operation when other experts turn out not “helpful” for the task  $t$ .

At each layer  $l$  in the ViT, for a new task  $t$ , the task-similarity oriented operation sampling is realized by a 2-level hierarchical sampling (Fig. 5 bottom):

- The first level uses a categorical distribution with the maximum number of entries being  $t$  consisting of at most the previous  $t-1$  tasks (some of which may use Skip and thus will be ignored) and the Aux expert. The categorical distribution  $(\psi_1, \dots, \psi_i, \dots, \psi_{I-1}, \psi_I)$  is computed by the Softmax function over the similarity scores defined above, where  $I \leq t$ .
- With a previous task  $i$  sampled with the probability  $\psi_i$ , at the second level of sampling, we sample the Reuse operation for the associated expert using a Bernoulli distribution with the success rate computed by the Sigmoid

function of the task similarity score defined by  $\rho_i = \frac{1}{1 + \exp(-S_i^{t,t})}$ , and the `Adapt` operation with probability  $1 - \rho_i$ .

### 3.4. Compute-Aware Target Network Selection

After the Supernet is trained, we propose a compute-sensitive evolutionary search on top of [32]. It first draws a population with a predefined number of candidate architectures from the trained Supernet using our proposed HEE sampling method. It then evolves the population via the crossover and the mutation operations. At each evolution iteration, the population is evaluated and sorted based on the trade-off between the validation performance and the compute of candidates: we predefine a performance tolerance threshold  $\tau$  (e.g.,  $\tau = 2\%$ ) to group candidate networks, and rank candidate networks in each group based on their compute in the increasing order. With the top- $k$  candidates after evaluation and sorting (the number  $k$  is predefined), for crossover, two randomly sampled candidate networks in the top- $k$  are crossed to produce a new target network. For mutation, a randomly selected candidate in the top- $k$  mutates its every choice block with probability (e.g., 0.1) to produce a new candidate. Crossover and mutation are repeated to generate sufficient new candidate target networks to form the population for the next iteration. We study the effect of varying the  $\tau$  in Fig. 10b in the supplementary. The final target network is retrained by randomly initializing all the learnable parameters following observations in [24].

### 3.5. Balancing Exploration and Exploitation

As illustrated in Fig. 5, to harness the best of the pure exploration strategy and the proposed exploitation strategy, we apply epoch-wise exploration and exploitation sampling for simplicity. For the pure exploration, we directly uniformly sample the experts at a layer  $l$ , consisting of the  $n$  experts from the previous  $t - 1$  tasks, and the `New` and `Skip` operations, where  $n \leq t - 1$ . At the beginning of an epoch in the Supernet training, we choose the pure exploration strategy with a probability of  $\epsilon_1$  (e.g., 0.3), and the hierarchical sampling strategy with a probability of  $1 - \epsilon_1$ . Similarly, when generating the initial population during the evolutionary search, we draw a candidate target network from a uniform distribution over the operations with a probability of  $\epsilon_2$ , and from the hierarchical sampling process with a probability of  $1 - \epsilon_2$ , respectively. In practice, we set  $\epsilon_2 > \epsilon_1$  (e.g.,  $\epsilon_2 = 0.5$ ) to encourage more exploration during the evolutionary search, while encouraging more exploitation for faster learning in the Supernet training. We study the effect of  $\epsilon_1$  and  $\epsilon_2$  in Fig. 10a in the supplementary.

## 4. Experiments

**Data.** We evaluate CHEEM on two challenging benchmarks, the MTIL benchmark [52] and the VDD bench-

mark [33], both consisting of tasks from varying domains with different complexities. VDD presents a significantly large class imbalance. For example, out of the total 2128 classes (excluding ImageNet-1k), Omniglot contains 1623 classes, whereas DTD contains only 47. Further details of the benchmarks can be found in the supplementary (Sec. H).

**Metrics.** We measure the performance of CHEEM using three metrics: **Average Accuracy**, **Average Forgetting** [4], and our proposed Figure-of-Merit (**FoM quantifying how many times a method is better than the other**). Let  $(\mathcal{F}_i, \mathcal{H}_i)$  be the feature backbone and the classifier heads after completion of task  $i$ , and  $a_{i,j} = \text{Acc}(D_j^{\text{test}}; \mathcal{F}_i, \mathcal{H}_i)$  be the Top-1 accuracy on the testing data for task  $j$  computed using  $(\mathcal{F}_i, \mathcal{H}_i)$ . The Average Accuracy ( $A\mathbb{A}$ ) and Average Forgetting ( $A\mathbb{F}$ ) are respectively defined as,

$$A\mathbb{A} = \frac{1}{N-1} \sum_{t=2}^N \text{Acc}(D_t^{\text{test}}; \mathcal{F}_N, \mathcal{H}_N), \quad (4)$$

$$A\mathbb{F} = \frac{1}{N-2} \sum_{t=2}^{N-1} \left( \max_{j \in [t, N]} a_{j,t} - a_{N,t} \right). \quad (5)$$

**FoM** explicitly and holistically compares two methods (e.g., our CHEEM against another baseline) with respect to their respective average accuracies and model complexities, where the model complexity is measured using FLOPs. For two methods  $m$  and  $n$ , we define the FoM as

$$\text{FoM}(m, n) = \frac{A\mathbb{A}^{\text{UpperBound}} - A\mathbb{A}^n}{A\mathbb{A}^{\text{UpperBound}} - A\mathbb{A}^m} \cdot \frac{\text{FLOPs}^n}{\text{FLOPs}^m}, \quad (6)$$

where  $A\mathbb{A}^{\text{UpperBound}}$  represents the average accuracy of upper-bound full task-to-task fine-tuning, and FLOPs is the computing cost. If a method  $m$  has smaller performance gap against the upper bound and smaller computing cost than another method  $n$ ,  $\text{FoM}(m, n)$  will be greater than 1. There is a trade-off between the first performance ratio and the second cost ratio. Intuitively,  $\text{FoM}(m, n)$  represents the relative magnitude of method  $m$  being better than  $n$ .

**Pretrained Models in ExfCCL.** We test two settings: one strong **ViT-Base** pretrained on the ImageNet-21k and fine-tuned on the ImageNet-1k, and the other relatively weaker **DEiT-Tiny** trained on ImageNet-1k. We report results of **CLIP ViT-Base** [31] in the supplementary (Sec. F).

**Implementation Details.** In all results reported in main text, we apply CHEEM to the  $\text{MLP}^{\text{Down}}$  layer, unless stated otherwise. We provide further implementation details in the supplementary (Sec. H).

**Baselines and Upper-Bounds** include:

- *Elastic Weight Consolidation* (EWC) [17].
- *State-of-the-art prompt-based methods*: CODA-Prompt [36], Dual-Prompt [43], Learning-to-Prompt (L2P) [44], S-Prompts [40] and DIKI [37].
- *Parameter-Efficient Fine-Tuning (PEFT) based continual learning*: LoRA [15] trained in a continual-learning setting, serving as an alternative internal-parameter memory

Table 1. FoM of CHEEM (MLP<sup>Down</sup>) against baselines on MTIL and VDD benchmarks.

| Method    | MTIL     |           | VDD      |           |
|-----------|----------|-----------|----------|-----------|
|           | ViT-Base | DEiT-Tiny | ViT-Base | DEiT-Tiny |
| EWC       | 10.5     | 26.0      | 12.1     | 678.9     |
| CODA-P    | 24.2     | 84.6      | 35.9     | 2811.3    |
| DualP     | 27.4     | 73.7      | 34.1     | 2130.6    |
| L2P       | 31.1     | 79.8      | 36.4     | 2431.6    |
| S-Prompts | 3.2      | 10.5      | 5.5      | 338.8     |
| DIKI      | 3.6      | 6.4       | 7.8      | 370.8     |
| LoRA-CL   | 1.7      | 5.7       | 1.5      | 73.6      |
| Tuna      | 50.6     | 242.4     | 58.6     | 4538.8    |
| Moal      | 2.38     | 7.64      | 8.63     | 462.8     |

applied to MLP<sup>down</sup> layer. This corresponds to a special case of CHEEM that uses the LoRA Adapt operator at every layer and omits NAS. We refer to this setting as LoRA-CL. We further compare with two recent methods based on PEFT - Moal [11] and Tuna [41].

- *Upper-bounds via task-to-task fine-tuning*: full task-to-task fine-tuning (UpperBound<sub>Full-FT</sub>) and LoRA-based task-to-task PEFT (UpperBound<sub>LoRA-FT</sub>), where the pre-trained model is independently fine-tuned on each task to provide an upper bound on continual-learning performance.

#### 4.1. FoM of CHEEM Against Baselines

Table 1 shows the FoM of CHEEM (how many times “better” CHEEM is) on MTIL and VDD benchmarks. **The consistently significant FoM shows that CHEEM can balance Average Accuracy and FLOPs**, whereas the baseline methods fall short on one or both of the axes. The special case of CHEEM, LoRA-CL is close to CHEEM in terms of FoM (1.7 on MTIL and 1.5 on VDD) when ViT-Base is used, but is significantly worse when DEiT-Tiny is used (5.7 on MTIL and 73.6 on VDD).

#### 4.2. CHEEM vs Upper-Bound T2T Fine-Tuning

Table 2 and Table 3 show the comparisons between our CHEEM and the two upper-bound fine-tuning methods. **Our CHEEM closely approaches full fine-tuning performance**. On MTIL, CHEEM achieves 85.9% vs 88.1% for ViT-Base, and 74.5% vs 75.3% for DEiT-Tiny. On VDD, CHEEM achieves 86.7% vs 88.7% for ViT-Base, and 76.2% vs 76.2% for DEiT-Tiny. We note that FLOPs of CHEEM are nearly doubled since the task ID inference uses an additional forward computation of the initial backbones. The same is for other prompting-based methods.

#### 4.3. Break-Down Comparisons with Baselines

Table 4 and Table 5 show the results. Although EWC uses the least FLOPs, it severely suffers from catastrophic forgetting due to the restriction of maintaining a single shared backbone, and can only reach average accuracy 44.58% for ViT-Base and 35.33% for DEiT-Tiny. Similarly, on both

Table 2. CHEEM vs Upper-Bounds on MTIL with three seeds.

| Method                        | ViT-Base   |            |       | DEiT-Tiny  |            |       |
|-------------------------------|------------|------------|-------|------------|------------|-------|
|                               | Avg. Acc   | Avg. Frgt. | FLOPs | Avg. Acc   | Avg. Frgt. | FLOPs |
| UpperBound <sub>Full-FT</sub> | 88.1 ± 0.0 | -          | 33.7  | 75.3 ± 0.1 | -          | 2.2   |
| UpperBound <sub>LoRA-FT</sub> | 87.4 ± 0.0 | -          | 33.7  | 74.6 ± 0.1 | -          | 2.2   |
| CHEEM                         | 85.9 ± 0.3 | 1.7 ± 0.1  | 62.3  | 74.5 ± 0.3 | 1.9 ± 0.0  | 4.5   |

Table 3. CHEEM vs Upper-Bounds on VDD with three seeds.

| Method                        | ViT-Base   |            |       | DEiT-Tiny   |            |       |
|-------------------------------|------------|------------|-------|-------------|------------|-------|
|                               | Avg. Acc   | Avg. Frgt. | FLOPs | Avg. Acc    | Avg. Frgt. | FLOPs |
| UpperBound <sub>Full-FT</sub> | 88.7 ± 0.1 | -          | 33.7  | 76.21 ± 0.1 | -          | 2.2   |
| UpperBound <sub>LoRA-FT</sub> | 86.8 ± 0.1 | -          | 33.7  | 76.30 ± 0.3 | -          | 2.2   |
| CHEEM                         | 86.7 ± 0.2 | 0.4 ± 0.0  | 61.6  | 76.18 ± 0.1 | 1.0 ± 0.0  | 4.5   |

MTIL and VDD, DIKI achieves lower FLOPs, but sacrifices performance. The special case of CHEEM, LoRA-CL achieves Average Accuracy close to CHEEM, but requires higher FLOPs as it cannot skip modules. The FoM of CHEEM against baselines for DEiT-Tiny are significantly large on VDD (Table 1) since CHEEM almost reaches the full fine-tuning performance (76.18% vs 76.21%), resulting in a very large accuracy gap ratio term in Eqn. 6.

Table 4. Comparison of Average Accuracy and Forgetting Rate on the MTIL benchmark with three seeds.

| Method    | ViT-Base   |            |       | DEiT-Tiny  |            |       |
|-----------|------------|------------|-------|------------|------------|-------|
|           | Avg. Acc   | Avg. Frgt. | FLOPs | Avg. Acc   | Avg. Frgt. | FLOPs |
| EWC       | 44.6 ± 6.4 | 23.8 ± 6.5 | 33.7  | 35.3 ± 0.3 | 7.3 ± 0.6  | 2.2   |
| CODA-P    | 40.2 ± 1.2 | 25.3 ± 1.8 | 70.3  | 5.6 ± 0.3  | 42.6 ± 0.8 | 5.0   |
| DualP     | 33.8 ± 0.4 | 22.1 ± 0.4 | 70.3  | 30.9 ± 0.3 | 17.5 ± 0.3 | 5.0   |
| L2P       | 26.6 ± 0.2 | 31.0 ± 0.3 | 70.3  | 23.2 ± 0.1 | 25.8 ± 0.4 | 5.1   |
| S-Prompts | 81.6 ± 0.4 | 1.6 ± 0.1  | 67.6  | 67.3 ± 0.4 | 1.8 ± 0.0  | 4.4   |
| DIKI      | 76.4 ± 0.0 | 2.0 ± 0.0  | 42.5  | 67.6 ± 0.1 | 1.8 ± 0.0  | 2.8   |
| LoRA-CL   | 84.7 ± 0.0 | 1.6 ± 0.1  | 68.2  | 71.1 ± 0.0 | 1.9 ± 0.0  | 4.5   |
| Tuna      | 70.7 ± 0.1 | 17.4 ± 0.2 | 405.0 | 44.5 ± 0.5 | 30.2 ± 0.3 | 26.0  |
| MoAL      | 78.3 ± 0.1 | 3.7 ± 0.1  | 33.7  | 63.8 ± 0.0 | 2.8 ± 0.0  | 2.2   |
| CHEEM     | 85.9 ± 0.3 | 1.7 ± 0.1  | 62.3  | 74.5 ± 0.3 | 1.9 ± 0.0  | 4.5   |

Table 5. Comparison of Average Accuracy and Forgetting Rate on the VDD benchmark with three seeds.

| Method    | ViT-Base   |            |       | DEiT-Tiny   |            |       |
|-----------|------------|------------|-------|-------------|------------|-------|
|           | Avg. Acc   | Avg. Frgt. | FLOPs | Avg. Acc    | Avg. Frgt. | FLOPs |
| EWC       | 44.0 ± 1.3 | 5.1 ± 1.1  | 33.7  | 33.7 ± 0.2  | 1.5 ± 0.1  | 2.2   |
| CODA-P    | 24.9 ± 2.2 | 26.1 ± 0.8 | 70.3  | 1.1 ± 0.1   | 37.6 ± 0.4 | 5.0   |
| DualP     | 28.1 ± 0.9 | 3.2 ± 0.5  | 70.3  | 19.4 ± 0.6  | 10.5 ± 0.5 | 5.0   |
| L2P       | 23.9 ± 0.7 | 9.0 ± 0.6  | 70.3  | 11.5 ± 0.8  | 20.9 ± 1.7 | 5.1   |
| S-Prompts | 78.6 ± 0.1 | 0.4 ± 0.0  | 67.6  | 65.8 ± 0.3  | 0.9 ± 0.0  | 4.4   |
| DIKI      | 65.9 ± 0.1 | 0.1 ± 0.0  | 42.5  | 58.3 ± 0.1  | 0.6 ± 0.0  | 2.8   |
| LoRA-CL   | 86.0 ± 0.1 | 0.3 ± 0.0  | 68.2  | 74.0 ± 0.3  | 1.1 ± 0.0  | 4.5   |
| Tuna      | 70.7 ± 0.1 | 14.9 ± 0.1 | 337.0 | 47.8 ± 0.2  | 22.9 ± 0.3 | 21.5  |
| MoAL      | 56.7 ± 0.1 | 1.0 ± 0.0  | 33.7  | 47.9 ± 0.1  | 2.1 ± 0.0  | 2.2   |
| CHEEM     | 86.7 ± 0.2 | 0.4 ± 0.0  | 61.6  | 76.18 ± 0.1 | 1.0 ± 0.0  | 4.5   |

#### 4.4. Importance of Designs in CHEEM

**Importance of structure updates to backbone - CHEEM vs Prompting-based Baselines.** Three prompting-based methods (CODA-Prompt, DualPrompt and L2P) perform even worse than EWC for both ViT-Base and DEiT-Tiny, mainly due to the discrepancy between global argmax vs. local argmax in their head classifier designs. CODA-Prompt almost completely fails for DEiT-Tiny with 5.62%

Table 6. Task-wise FLOPs of CHEEM on MTIL using ViT-Base.

| Airc           | C101           | CIFAR          | DTD            | ESAT           | Flwr           |
|----------------|----------------|----------------|----------------|----------------|----------------|
| 62.8 $\pm$ 0.9 | 59.0 $\pm$ 0.9 | 64.0 $\pm$ 0.0 | 64.6 $\pm$ 0.9 | 57.8 $\pm$ 0.9 | 62.1 $\pm$ 0.1 |
| F101           | MNIST          | Pets           | Cars           | SUN397         | Avg            |
| 65.2 $\pm$ 1.7 | 56.0 $\pm$ 0.9 | 62.8 $\pm$ 0.9 | 65.9 $\pm$ 1.5 | 65.2 $\pm$ 0.9 | 62.3 $\pm$ 0.3 |

Table 7. Average training time per task (in hours) on MTIL. The total training time of CHEEM (1.45h) is split as: Supernet - 0.6h, Search - 0.53h, Finetuning - 0.32h.

| CHEEM | CODA-P | L2P  | Dual-P | SPrompts | DIKI | LoRA-CL |
|-------|--------|------|--------|----------|------|---------|
| 1.45  | 1.26   | 1.26 | 1.25   | 0.38     | 0.38 | 0.38    |

average accuracy. S-Prompts works the best among prompting based methods, but is still inferior to our CHEEM: 4% drop for ViT-Base, and 7% drop for DEiT-Tiny. This shows the importance of inferring task IDs on the fly for streaming tasks with significantly varying distributions of classes. Overall, *the superior performance of CHEEM shows the importance of structurally and dynamically updating the backbone with the task-synergy internal memory.*

**Importance of Search - CHEEM vs LoRA-CL.** Both are applied to MLP<sup>Down</sup> and use the same external task-centroid memory for task IDs inference. The improvement by CHEEM, 1% increase for ViT-Base and 3.45% increase for DEiT-Tiny show the benefits of HEE-NAS, *especially for weaker backbones such as DEiT-Tiny, leading to more competent ExfCCL that is less sensitive to start backbone.*

#### 4.5. CHEEM Is Unique for Task-Difficulty Awareness Using ViTs in ExfCCL

Intuitively, easier tasks should require lesser FLOPs in continual learning. Table 6 shows that CHEEM allocates lower FLOPs to easier tasks like MNIST and ESAT. Figs. 3b and 3c show some examples of architectures learned by CHEEM on the MTIL benchmark. **These sensible model structures are unique to our CHEEM in comparisons to other baselines.** They also show interesting yet “irregular” model configurations caused by learned Skip operations in different blocks in Fig. 3b: two consecutive Transformer blocks with one block comprising only the attention component (for token mixing) without the FFN (for channel mixing). Fig. 6 in the supplementary shows the sensible model structures learned by CHEEM on VDD.

#### 4.6. Reasonable Training Overhead of CHEEM

The NAS overhead in CHEEM leads to marginal increase in total training time relative to L2P, DualPrompt, or CODA-Prompt. While it is more expensive than SPrompts, DIKI, and LoRA-CL, CHEEM lowers inference FLOPs compared to SPrompts and LoRA-CL and achieves higher average accuracy (Tables 4 and 5). As shown in Table 7, the full CHEEM pipeline (supernet training, search, and finetuning) remains competitive with baseline methods, offering a practical approach for learning task-specific backbones.

#### 4.7. Ablation Studies

• **CHEEM<sub>lite</sub> using lightweight models for task ID learning in the external memory:** CHEEM can use a lightweight pretrained backbone for task ID learning, rather than the backbone in adaptation (Section G), effectively making the FLOPs overhead negligible while outperforming the baselines.

• **CHEEM placement: MLP<sup>Down</sup> vs. Projection.** Table 8 shows the comparisons. Both CHEEM achieve on-par average accuracy performance. However, due to the size of the FFN block, when skipping the FFN block rather than the MHSA block, CHEEM (MLP<sup>Down</sup>) shows better FLOPs reduction.

Table 8. Comparisons of two selected CHEEM placements.

| Dataset | Method              | ViT-Base     |       |        | DEiT-Tiny    |       |        |
|---------|---------------------|--------------|-------|--------|--------------|-------|--------|
|         |                     | Avg. Acc     | FLOPs | %Param | Avg. Acc     | FLOPs | %Param |
| MTIL    | MLP <sup>Down</sup> | <b>85.88</b> | 62.31 | 0.40   | <b>74.51</b> | 4.47  | 6.32   |
|         | Attn Proj           | 85.57        | 65.91 | 0.25   | 74.39        | 4.42  | 1.79   |

• **Sampling in NAS: HEE vs. Uniform.** Table 9 shows the comparisons. Although both methods achieve on-par average accuracy, the uniform sampling method leads to much higher number of new parameter increase (due to New and Adapt): 5.89% vs 0.25% for ViT-Base (see Figure 8 in the supplementary), and 14.93% vs 6.32% for DEiT-Tiny. *The promising performance of uniform sampling based NAS shows the representational power of our proposed internal parameter memory using the four basic operations (Reuse, Adapt, New and Skip). The parsimoniousness of HEE-NAS highlights its efficacy in continual learning by effectively leveraging task synergies.*

Table 9. Comparisons of HEE and Uniform (i.e., Pure Exploration) sampling during Supernet training on MTIL benchmark.

| Method  | ViT-Base     |       |        | DEiT-Tiny    |       |        |
|---------|--------------|-------|--------|--------------|-------|--------|
|         | Avg. Acc     | FLOPs | %Param | Avg. Acc     | FLOPs | %Param |
| HEE     | <b>85.88</b> | 62.31 | 0.25   | 74.51        | 4.47  | 6.32   |
| Uniform | 84.74        | 61.82 | 5.89   | <b>75.05</b> | 4.47  | 14.93  |

• **Effect of task orders.** In Table 11 in the supplementary, we show that CHEEM is insensitive to task order.

#### 5. Conclusion

This paper presents a method of transforming Vision Transformers (ViTs) for exemplar-free class-incremental continual learning (ExfCCL), dubbed **CHEEM** (Continual Hierarchical-Exploration-Exploitation Memory). The core of CHEEM is its internal (parameter) memory, which is realized by a proposed Hierarchical-Exploration-Exploitation (HEE) sampling based neural architecture search algorithm. CHEEM is tested on two challenging benchmarks, the MTIL and VDD benchmarks. It obtains state-of-the-art performance on both benchmarks, outperforming the prior art by a large margin, with sensible CHEEM structures continually learned.



## Acknowledgments

This work was supported in part by ARO Grants W911NF1810295 and W911NF2210010, NSF awards IIS-1909644, CMMI-2024688, and IUSE-2013451, as well as the NC State Goodnight Early Career Award. Portions of M. Dai's contributions were completed while she was an undergraduate student at Princeton University. The views and conclusions expressed in this paper are those of the authors and do not necessarily reflect the official policies or endorsements, either expressed or implied, of ARO, NSF, or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for governmental purposes notwithstanding any copyright notation herein.

## References

- [1] Rahaf Aljundi, Punarjay Chakravarty, and Tinne Tuytelaars. Expert gate: Lifelong learning with a network of experts. In *2017 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017, Honolulu, HI, USA, July 21-26, 2017*, pages 7120–7129. IEEE Computer Society, 2017. 4
- [2] Rahaf Aljundi, Francesca Babiloni, Mohamed Elhoseiny, Marcus Rohrbach, and Tinne Tuytelaars. Memory aware synapses: Learning what (not) to forget. In *Computer Vision - ECCV 2018 - 15th European Conference, Munich, Germany, September 8-14, 2018, Proceedings, Part III*, pages 144–161. Springer, 2018. 4
- [3] Rahaf Aljundi, Marcus Rohrbach, and Tinne Tuytelaars. Selfless sequential learning. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019. 4
- [4] Arslan Chaudhry, Puneet Kumar Dokania, Thalaiyasingam Ajanthan, and Philip H. S. Torr. Riemannian walk for incremental learning: Understanding forgetting and intransigence. In *Computer Vision - ECCV 2018 - 15th European Conference, Munich, Germany, September 8-14, 2018, Proceedings, Part XI*, pages 556–572. Springer, 2018. 6
- [5] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. 1, 2, 4
- [6] Arthur Douillard, Matthieu Cord, Charles Ollion, Thomas Robert, and Eduardo Valle. Podnet: Pooled outputs distillation for small-tasks incremental learning. In *Computer Vision - ECCV 2020 - 16th European Conference, Glasgow, UK, August 23-28, 2020, Proceedings, Part XX*, pages 86–102. Springer, 2020. 4
- [7] Arthur Douillard, Alexandre Ramé, Guillaume Couairon, and Matthieu Cord. Dytox: Transformers for continual learning with dynamic token expansion. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022*, pages 9275–9285. IEEE, 2022. 4
- [8] Beyza Ermis, Giovanni Zappella, Martin Wistuba, Aditya Rawal, and Cédric Archambeau. Continual learning with transformers for image classification. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, CVPR Workshops 2022, New Orleans, LA, USA, June 19-20, 2022*, pages 3773–3780. IEEE, 2022. 2, 4
- [9] Chrisantha Fernando, Dylan Banarse, Charles Blundell, Yori Zwols, David Ha, Andrei A. Rusu, Alexander Pritzel, and Daan Wierstra. Pathnet: Evolution channels gradient descent in super neural networks. *CoRR*, abs/1701.08734, 2017. 4
- [10] Qiankun Gao, Chen Zhao, Yifan Sun, Teng Xi, Gang Zhang, Bernard Ghanem, and Jian Zhang. A unified continual learning framework with general parameter-efficient tuning. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*, pages 11449–11459. IEEE, 2023. 4
- [11] Zijian Gao, Wangwang Jia, Xingxing Zhang, Dulan Zhou, Kele Xu, Feng Dawei, Yong Dou, Xinjun Mao, and Huaimin Wang. Knowledge memorization and rumination for pre-trained model-based class-incremental learning. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 20523–20533, 2025. 7
- [12] Yunhao Ge, Yuecheng Li, Shuo Ni, Jiaping Zhao, Ming-Hsuan Yang, and Laurent Itti. CLR: channel-wise lightweight reprogramming for continual learning. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*, pages 18752–18762. IEEE, 2023. 2
- [13] Yunhao Ge, Yuecheng Li, Di Wu, Ao Xu, Adam M. Jones, Amanda Sofie Rios, Iordanis Fostiropoulos, shixian wen, Po-Hsuan Huang, Zachary William Murdock, Gozde Sahin, Shuo Ni, Kiran Lekkala, Sumedh Anand Sontakke, and Laurent Itti. Lightweight learner for shared knowledge lifelong learning. *Transactions on Machine Learning Research*, 2023. 2
- [14] Zichao Guo, Xiangyu Zhang, Haoyuan Mu, Wen Heng, Zechun Liu, Yichen Wei, and Jian Sun. Single path one-shot neural architecture search with uniform sampling. In *Computer Vision - ECCV 2020 - 16th European Conference, Glasgow, UK, August 23-28, 2020, Proceedings, Part XVI*, pages 544–560. Springer, 2020. 5
- [15] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. 3, 5, 6
- [16] Ahmet Iscen, Thomas Bird, Mathilde Caron, Alireza Fathi, and Cordelia Schmid. A memory transformer network for incremental learning. In *33rd British Machine Vision Conference 2022, BMVC 2022, London, UK, November 21-24, 2022*, page 388. BMVA Press, 2022. 4
- [17] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic for-

- getting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. 4, 6
- [18] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. 4
- [19] Duo Li, Guimei Cao, Yunlu Xu, Zhanzhan Cheng, and Yi Niu. Technical report for ICCV 2021 challenge sslad-track3b: Transformers are better continual learners. *CoRR*, abs/2201.04924, 2022. 4
- [20] Xilai Li, Yingbo Zhou, Tianfu Wu, Richard Socher, and Caiming Xiong. Learn to grow: A continual structure learning framework for overcoming catastrophic forgetting. In *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, pages 3925–3934. PMLR, 2019. 1, 4
- [21] Zhizhong Li and Derek Hoiem. Learning without forgetting. *IEEE Trans. Pattern Anal. Mach. Intell.*, 40(12):2935–2947, 2018. 4
- [22] Yan-Shuo Liang and Wu-Jun Li. Inflora: Interference-free low-rank adaptation for continual learning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2024, Seattle, WA, USA, June 16-22, 2024*, pages 23638–23647. IEEE, 2024. 2
- [23] Hanxiao Liu, Karen Simonyan, and Yiming Yang. DARTS: differentiable architecture search. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019. 4
- [24] Zhuang Liu, Mingjie Sun, Tinghui Zhou, Gao Huang, and Trevor Darrell. Rethinking the value of network pruning. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019. 6
- [25] Arun Mallya, Dillon Davis, and Svetlana Lazebnik. Piggyback: Adapting a single network to multiple tasks by learning to mask weights. In *Computer Vision - ECCV 2018 - 15th European Conference, Munich, Germany, September 8-14, 2018, Proceedings, Part IV*, pages 72–88. Springer, 2018. 2
- [26] Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. pages 109–165. Academic Press, 1989. 1
- [27] Abdelrahman Mohamed, Rushali Grandhe, K. J. Joseph, Salman H. Khan, and Fahad Shahbaz Khan. D<sup>3</sup>former: Debaised dual distilled transformer for incremental learning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2023 - Workshops, Vancouver, BC, Canada, June 17-24, 2023*, pages 2421–2430. IEEE, 2023. 4
- [28] Pedro Morgado and Nuno Vasconcelos. Nettailor: Tuning the architecture, not just the weights. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2019, Long Beach, CA, USA, June 16-20, 2019*, pages 3044–3054. Computer Vision Foundation / IEEE, 2019. 2
- [29] Cuong V. Nguyen, Yingzhen Li, Thang D. Bui, and Richard E. Turner. Variational continual learning. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net, 2018. 4
- [30] Francesco Pelosin, Saurav Jha, Andrea Torsello, Bogdan Raducanu, and Joost van de Weijer. Towards exemplar-free continual learning in vision transformers: an account of attention, functional and weight regularization. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, CVPR Workshops 2022, New Orleans, LA, USA, June 19-20, 2022*, pages 3819–3828. IEEE, 2022. 4
- [31] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, pages 8748–8763. PMLR, 2021. 6
- [32] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V. Le. Regularized evolution for image classifier architecture search. In *The Thirty-Third AAAI Conference on Artificial Intelligence, AAAI 2019, The Thirty-First Innovative Applications of Artificial Intelligence Conference, IAAI 2019, The Ninth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2019, Honolulu, Hawaii, USA, January 27 - February 1, 2019*, pages 4780–4789. AAAI Press, 2019. 6
- [33] Sylvestre-Alvise Rebuffi, Hakan Bilen, and Andrea Vedaldi. Learning multiple visual domains with residual adapters. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pages 506–516, 2017. 1, 4, 6
- [34] Andrei A. Rusu, Neil C. Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks. *CoRR*, abs/1606.04671, 2016. 4
- [35] Jonathan Schwarz, Wojciech Czarnecki, Jelena Luketina, Agnieszka Grabska-Barwinska, Yee Whye Teh, Razvan Pascanu, and Raia Hadsell. Progress & compress: A scalable framework for continual learning. In *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10-15, 2018*, pages 4535–4544. PMLR, 2018. 4
- [36] James Seale Smith, Leonid Karlinsky, Vyshnavi Gutta, Paola Cascante-Bonilla, Donghyun Kim, Assaf Arbelle, Rameswar Panda, Rogério Feris, and Zsolt Kira. Coda-prompt: Continual decomposed attention-based prompting for rehearsal-free continual learning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2023, Vancouver, BC, Canada, June 17-24, 2023*, pages 11909–11919. IEEE, 2023. 1, 2, 4, 6
- [37] Longxiang Tang, Zhuotao Tian, Kai Li, Chunming He, Hantao Zhou, Hengshuang Zhao, Xiu Li, and Jiaya Jia. Mind the interference: Retaining pre-trained knowledge in parameter efficient continual learning of vision-language models. In

- Computer Vision - ECCV 2024 - 18th European Conference, Milan, Italy, September 29-October 4, 2024, Proceedings, Part XXXVI*, pages 346–365. Springer, 2024. [4](#), [6](#)
- [38] Sebastian Thrun and Tom M. Mitchell. Lifelong robot learning. *Robotics Auton. Syst.*, 15(1-2):25–46, 1995. [1](#)
- [39] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, pages 10347–10357. PMLR, 2021. [2](#), [4](#)
- [40] Yabin Wang, Zhiwu Huang, and Xiaopeng Hong. S-prompts learning with pre-trained transformers: An occam’s razor for domain incremental learning. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022. [1](#), [2](#), [3](#), [4](#), [6](#)
- [41] Yan Wang, Da-Wei Zhou, and Han-Jia Ye. Integrating task-specific and universal adapters for pre-trained model-based class-incremental learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 806–816, 2025. [4](#), [7](#)
- [42] Zhen Wang, Liu Liu, Yiqun Duan, Yajing Kong, and Dacheng Tao. Continual learning with lifelong vision transformer. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022*, pages 171–181. IEEE, 2022. [4](#)
- [43] Zifeng Wang, Zizhao Zhang, Sayna Ebrahimi, Ruoxi Sun, Han Zhang, Chen-Yu Lee, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer G. Dy, and Tomas Pfister. Dualprompt: Complementary prompting for rehearsal-free continual learning. In *Computer Vision - ECCV 2022 - 17th European Conference, Tel Aviv, Israel, October 23-27, 2022, Proceedings, Part XXVI*, pages 631–648. Springer, 2022. [1](#), [2](#), [4](#), [6](#)
- [44] Zifeng Wang, Zizhao Zhang, Chen-Yu Lee, Han Zhang, Ruoxi Sun, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer G. Dy, and Tomas Pfister. Learning to prompt for continual learning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022*, pages 139–149. IEEE, 2022. [1](#), [2](#), [4](#), [6](#)
- [45] Mitchell Wortsman, Vivek Ramanujan, Rosanne Liu, Aniruddha Kembhavi, Mohammad Rastegari, Jason Yosinski, and Ali Farhadi. Supermasks in superposition. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020. [2](#)
- [46] Yichen Wu, Hongming Piao, Long-Kai Huang, Renzhen Wang, Wanhua Li, Hanspeter Pfister, Deyu Meng, Kede Ma, and Ying Wei. Sd-lora: Scalable decoupled low-rank adaptation for class incremental learning. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. [2](#)
- [47] Mengqi Xue, Haofei Zhang, Jie Song, and Mingli Song. Meta-attention for vit-backed continual learning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022*, pages 150–159. IEEE, 2022. [2](#), [4](#)
- [48] Jaehong Yoon, Eunho Yang, Jeongtae Lee, and Sung Ju Hwang. Lifelong learning with dynamically expandable networks. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net, 2018. [4](#)
- [49] Jiazuo Yu, Yunzhi Zhuge, Lu Zhang, Ping Hu, Dong Wang, Huchuan Lu, and You He. Boosting continual learning of vision-language models via mixture-of-experts adapters. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 23219–23230, 2024. [4](#)
- [50] Pei Yu, Yinpeng Chen, Ying Jin, and Zicheng Liu. Improving vision transformers for incremental learning. *CoRR*, abs/2112.06103, 2021. [4](#)
- [51] Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. In *Proceedings of the 34th International Conference on Machine Learning, ICML 2017, Sydney, NSW, Australia, 6-11 August 2017*, pages 3987–3995. PMLR, 2017. [4](#)
- [52] Zangwei Zheng, Mingyuan Ma, Kai Wang, Ziheng Qin, Xiangyu Yue, and Yang You. Preventing zero-shot transfer degradation in continual learning of vision-language models. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*, pages 19068–19079. IEEE, 2023. [1](#), [3](#), [4](#), [6](#)